

Guia Técnico Avançado: Pré-treinamento de um LLM do Zero

Arquitetura de ~250 Milhões de Parâmetros Otimizada para Língua Portuguesa

1. Introdução ao Pré-treinamento Causal (From-Scratch)

Diferente do ajuste fino (fine-tuning), onde adaptamos um cérebro já instruído, o **pré-treinamento do zero** força o modelo a decifrar a estrutura de uma língua por meio da pura probabilidade estatística. O modelo começa com pesos inicializados aleatoriamente, gerando ruído textual, e progressivamente aprende morfologia, sintaxe, semântica e contexto.

O objetivo principal é a modelagem de linguagem causal (Causal Language Modeling - CLM), ou seja, prever o próximo token dado um contexto anterior de tamanho fixo.

2. Dimensionamento dos Hiperparâmetros (~250M Parâmetros)

Para atingir a marca de aproximadamente 250 milhões de parâmetros utilizando a arquitetura clássica do GPT-2, configuramos os seguintes hiperparâmetros:

Hiperparâmetro	Valor Configurado	Impacto e Descrição
Tamanho do Vocabulário (V)	32.000 tokens	Otimizado para o Português (BPE do zero)
Dimensão Oculta (d_model)	1.024	Tamanho dos vetores de embedding interno
Camadas de Transformer (N)	16 blocos	Profundidade para processar semântica complexa
Cabeças de Atenção (h)	16 cabeças	Dimensão de cada cabeça = $d_model / h = 64$
Contexto Máximo (T)	1.024 tokens	Janela máxima de atenção causal concorrente

Equação de Parâmetros Estimada:

A quantidade de parâmetros em um decoder Transformer cresce aproximadamente seguindo a fórmula:

$$\text{Parâmetros} \approx V \times d_model + N \times (12 \times d_model^2)$$

Substituindo nossos valores: $32.000 \times 1.024 + 16 \times (12 \times 1.024^2) \approx 32,7M + 201,3M \approx$ **234 Milhões de parâmetros** (excluindo biases e camadas de normalização finais, que elevam o total para ~250M).

3. Tokenização e Pipeline de Dados Avançado

O uso do Tokenizer do GPT-2 padrão do inglês degradaria severamente a performance em português, pois fragmentaria palavras comuns em múltiplos caracteres (ex: 'computação' viraria 4 tokens isolados). Treinar um Tokenizer BPE (Byte Pair Encoding) do zero garante máxima eficiência de memória e compressão semântica.

- **Streaming de Dados:** Como o dump da Wikipédia em português ultrapassa os gigabytes de texto plano, o uso de iteradores sob demanda (`streaming=True`) evita o esgotamento da memória RAM do ambiente de execução do Colab.
- **Data Collator Causal:** Garante que os rótulos de treino sejam cópias exatas dos inputs deslocadas em uma posição para a direita, essencial para o aprendizado auto-regressivo.

4. Mitigação de Gargalos no Google Colab (VRAM)

Para evitar o erro fatal de falta de memória de vídeo (**CUDA Out of Memory**), o pipeline incorpora duas técnicas essenciais de engenharia de machine learning:

1. **Precisão Mista de Ponto Flutuante (FP16):** Em vez de usar precisão de 32 bits (`float32`) para todos os pesos e gradientes, ativamos o FP16. Isso reduz o consumo de memória VRAM pela metade e aproveita os Tensor Cores das GPUs modernas (como T4, L4 ou A100).
2. **Acumulação de Gradientes (Gradient Accumulation):** Atualizar os parâmetros com um lote real muito grande estoura a GPU. Configurando o lote por dispositivo em 2 (`per_device_train_batch_size=2`) e acumulando gradientes por 16 passos (`gradient_accumulation_steps=16`), simulamos perfeitamente um tamanho de lote global efetivo de 32, sem penalidade de memória.

5. Agendador de Taxa de Aprendizado (Learning Rate Scheduler)

Utilizamos o decaimento por Cosseno com uma fase inicial de aquecimento (Cosine Decay with Warmup):

- **Warmup Steps (2000 passos):** A taxa de aprendizado começa em 0 e sobe linearmente até $6e-4$. Isso impede que os gradientes massivos e caóticos das primeiras iterações destruam a inicialização do modelo.
- **Cosine Decay:** Após o pico, a taxa de aprendizado decai seguindo uma curva de cosseno até valores próximos de zero na iteração final, permitindo um refinamento cirúrgico dos parâmetros.

Persistência no Google Drive: O script realiza salvamentos em intervalos regulares (`save_steps=1000`) limitando os arquivos retidos de forma circular (`save_total_limit=2`). Isso assegura que, em caso de desconexão forçada da instância do Google Colab, o progresso possa ser retomado exatamente do último checkpoint disponível.